智能体动态记忆管理系统

最终报告

软件需求工程 | 团队：喵喵喵喵喵 | 成员：胡海洋 石子吟 廖若言 路巨聪 叶首状 | 2026-11-09

1. 项目概述

1.1. 目标与问题

长期多轮对话中，人物的计划、偏好与状态会随时间变化。全文阅读成本高，普通向量检索又可能同时取回新旧事实，使模型依据过时信息作答。本项目面向 LoCoMo 长对话，构建可检索、可追溯的结构化 Memory，在作答前判断哪些记忆仍然有效，并只依据有效证据生成带引用的答案；证据不足或冲突无法消解时拒绝猜测。

中期发现的核心问题是：固定 Top-K 只保证候选与问题“相关”，不保证同一主体、同一属性的完整状态历史都进入候选集，而状态解析只能处理已召回的 Memory。为此，最终版本在“初始检索”之后增加一轮补选：先实现 SRS 中 S 级需求 FR-RET-07 / FR-STS-09 的查询驱动同槽位补检，再将其扩展为显式建图的 Graph Memory 多跳补选，并改为仅在证据不足时触发的自适应策略。

1.2. 交付范围

交付物包括：结构化 Memory 构建与校验、BM25 / Embedding / Hybrid 检索、查询驱动补检与 Graph Memory、规则化动态关系分析、DeepSeek 作答与引用校验、单用户 Web 原型，以及可复现的评测脚本与结果包。系统不面向开放域数据或生产部署，不自动裁决真假，不提供 Memory 压缩与开放式因果推理。

> 实验共两次运行：11 月 5 日（v1，旧版答案提示词）与 11 月 9 日（v2，最终提示词）。两次使用完全相同的题单、原始对话、冻结 Memory 与检索代码。方法之间只在同一次运行内比较；跨运行只做同题配对的“提示词版本效应”分析。

2. 需求实现情况

下表按 SRS 的功能模块汇总最终实现与验收结果。中期标注为“未实现”的局部历史扩展与答案 F1 对照，均已在最终版本完成。

模块	SRS 编号	最终实现	验收结果
Memory 构建	FR-MEM-01 – 05	按 Schema 抽取主体 / 属性 / 值、观测时间与生效时间、来源 session / turn；6 个冻结样本共 2,882 条 Memory。	Schema 校验全部通过；每条 Memory 可回链原文。
初始检索	FR-RET-01 – 06	BM25、Embedding、Hybrid（BM25 0.4 + all-MiniLM-L6-v2 0.6）三种检索器，统一接口与降级。	Top-K 排序、分数与来源可检查。
补选（历史扩展）	FR-RET-07、FR-STS-09	查询驱动同槽位补检；Graph Memory 有界多跳补选；自适应触发。	已实现（中期为 S 级未实现）；见第 4 章。
状态与答案	FR-STS-01 – 08	RETAIN / UPDATE / SUPERSEDE / CONFLICT 及理由；DeepSeek 只依据候选证据作答。	冲突或证据不足时返回 null；引用 ID 严格校验。
服务接口	FR-API-01 – 05	FastAPI + Pydantic；`/api/query` 增加 Graph 参数与路径字段，JSON Schema 为唯一契约。	契约与 API 测试通过。
Web 展示	FR-UI-01 – 07	展示候选、扩展来源、状态关系、Graph 路径、答案、Memory ID 与原始证据。	真实 DeepSeek 查询完整证据链可复核。
评测	FR-EVL-01 – 06	证据召回、答案 F1 / EM、Token 与拒答统计；两次运行、七种方法、测试集与开发集的逐题配对对照。	Python 全量测试 198 项通过；前端构建通过。

质量属性与约束同样按 SRS 落实，重点是“答案可追溯”和“评测不泄漏”：

质量属性 / 约束	落实方式
可追溯性	每个非空答案必须引用当前候选中的 Memory ID，并可回链到原始 session / turn 与原文；Graph 新增候选附带完整路径。
可信性	证据不足、冲突无法消解或引用无效时返回 null，不使用模板答案兜底；拒答与错误分别计数，均计 0 分。
评测隔离（CON-02）	抽取、检索、建图与遍历都不读取 QA 标准答案、参考证据或评测标签；标准答案只在生成结束后用于本地评分。
可复现性	冻结题单、Memory 与提示词并记录 SHA-256；外部请求逐条 checkpoint，可中断续跑；一条脚本复现最终实验。
安全	模型密钥只从环境变量或 `.env` 读取，不写入代码、日志、结果或报告。

3. 系统设计与关键实现

3.1. 总体链路

LoCoMo 对话 → Memory 抽取 / 校验 → Hybrid Top-K → 自适应补选 → 关系与时间规则 → 证据包 → DeepSeek 作答 → 引用校验 / Web

Memory 构建、检索、建图、遍历、规则分析和引用校验都在本地完成；只有 Memory 抽取与最终作答调用大模型。前后端分别使用 FastAPI / Pydantic 与 React / TypeScript / Vite，共用严格的 JSON 契约。

3.2. Memory 与动态关系

Memory 以可独立检索的事实为粒度，记录 `memory_id`、`subject` / `attribute` / `value`、事实描述、对话观测时间与事件生效时间（二者不自动等同），以及来源 session、turn 和原文。解析器按“主体 + 属性”状态槽位比较 Memory，明确时间优先；旧 Memory 不删除而标记为历史或冲突，集合类事实不机械互斥。规则选出的 Effective Memory 只是可审计的提示，与 LLM 最终引用的 Memory 分层展示，不混为一谈。

字段	含义
`memory_id`	唯一且可复现的标识
`subject` / `attribute`	主体与可比较属性，构成状态槽位
`value` / `content`	结构化取值与可独立检索的事实描述
`time`	对话观测时间与事件生效时间分开记录
`evidence`	来源 session、turn（`dia_id`）与原文片段

关系	处理
RETAIN	后续信息确认原事实仍成立，保留为有效
UPDATE	补充兼容的新内容，合并为当前状态
SUPERSEDE	新状态使旧状态成为历史，旧条目标记 historical
CONFLICT	互不相容且先后无法判断，标记 conflicting，保守作答

3.3. 补选：查询驱动补检与 Graph Memory

查询驱动补检从 Top-K 种子识别主体与属性，补全同一槽位的历史状态；超出预算、无可用种子或缺少状态锚点时保守停止。

Graph Memory 只依据结构化 Memory 及其来源字段建图，不读取标准答案。节点为 memory、subject、slot、value、session、turn 六类；边包括 ABOUT、IN_SLOT、HAS_VALUE、OBSERVED_IN、SUPPORTED_BY、NEXT_STATE（同槽位按时间相邻）与 NEXT_TURN（同会话相邻轮次）。查询时从 Hybrid Top-K 出发做有界 best-first 遍历，最多 4 跳、新增不超过 8 条；候选按 25% 路径先验与 75% 问题词相关性排序，并以命名实体范围、集合槽位主题过滤和通用词过滤阻止跨人物或无关连边。每条新增候选返回路径与分数，并在证据包中标记为 `graph_supplemental`。

边	连接	在补选中的作用
NEXT_STATE	同槽位相邻 Memory	按时间串起状态链，可直接扩展，是找回“旧状态 / 新状态”的主要路径
IN_SLOT / ABOUT	Memory → 槽位 / 主体	把同一人物、同一属性的记忆聚到一起，限定补选范围
HAS_VALUE	Memory → 取值	关联取值相同的记忆，用于列表类与集合类问题
OBSERVED_IN / SUPPORTED_BY	Memory → 会话 / 轮次	回链来源，并经由证据轮次找到同一段对话中的相关记忆
NEXT_TURN	同会话相邻轮次	补回上下文相邻的证据；需通过问题词相关性过滤

图在导出时保留边的方向，遍历时允许双向查找证据。Graph 不推断原数据中不存在的因果边，也不自动裁决真假，保证每条路径都可复核、成本可控。

3.4. 自适应作答与引用校验

中期后的关键变化是不再对每题强制扩图：系统先用 Top-K 作答，只有模型因证据不足拒答，或问题明确询问多个名字时，才启动 Graph 补选重试；复数姓名题的新答案只在引用了更多 Memory 且确实用到图证据时才覆盖原答案。答案模型输出最短但完整的答案片段与支持它的 Memory ID：时间、时长、人物类问题保持所问粒度，列表题不得漏项。无效 ID、空引用和字符串 “null” 在边界层校验或归一化，不使用模板答案兜底。

中期状态	最终改进
固定 Top-K，状态解析无法恢复候选外证据	同槽位补检 + Graph 多跳补选
一律扩图时，简单题受无关证据干扰	仅在拒答或复数姓名题时自适应重试
长句与额外解释拉低 token F1	最短完整答案 + 列表完整性约束
候选、规则建议与答案引用容易混淆	三者分层展示，引用 ID 严格校验

3.5. 接口与前端

`/api/query` 在原有 `sample_id`、`question`、`top_k` 之外新增 `graph_expand`、`graph_retry_on_refusal`、`graph_max_hops`（1 – 4）与 `max_graph_memories`（1 – 50）等参数，参数越界直接返回 422；响应中的每条候选新增 `candidate_source`、`graph_hops`、`graph_path`，并在 `graph.paths` 中给出完整边类型。Graph 默认关闭，关闭时 `/api/query` 与 `/api/retrieve` 返回的 Top-K ID 及顺序与原系统完全一致。前端新增 Graph Memory 面板，按跳数展示补选路径；检索表格同时显示 Graph 排序分数与答案引用标记，提问范围限定在当前选中的样本内。

4. 实验与验证

4.1. 实验设计

项目	配置
数据划分	自定开发集 conv-41/42/43/44（650 题）；测试集 conv-47/48（340 题）；均为 category 1 – 4 QA。
两次运行	v1：旧版提示词，最大输出 1024 token，DeepSeek Flash 与 GLM-4-Flash。v2：新版提示词，最大输出 128 token，DeepSeek Flash，温度 0。
对照方法	Full Text（整段对话）；Embedding K5；Hybrid K5（BM25 0.4 + all-MiniLM-L6-v2 0.6）；K5 + 补检；Hybrid K10；Forced Graph（每题扩图）；Adaptive Graph（按需扩图）。Graph 最多 4 跳、新增 8 条。
指标	英文 token F1、EM、Memory 证据召回、平均候选数、答案模型平均输入 Token、拒答与错误数；方法差异用逐题配对 bootstrap（5,000 次）给出 95% 置信区间。

两次运行的题单、原始对话、冻结 Memory 以及检索、补检、关系规则代码的 SHA-256 完全一致，差异只在答案提示词、输出上限与模型。标准答案只在检索与生成全部完成后用于评分；每次外部请求立即写入 checkpoint，可中断续跑，密钥不写入任何结果。

4.2. 功能验证

检查项	结果
Python 全量测试（含 Graph / API / Schema 专项）	198 项通过
前端 TypeScript 检查与 Vite 生产构建	通过；`npm audit` 中等以上漏洞为 0
Demo 离线验收	8 / 8 通过（测试替身，只验证集成，不代表模型 F1）
Graph 参数边界矩阵	3 种检索器 × 4 种跳数 × 3 种预算，共 12,240 次题目配置运行全部通过：不超预算、无重复新增、每条新增都有路径、召回逐题不低于固定 K
6 个样本完整导出与真实 API 请求	节点 ID 唯一、边端点均存在并通过 Schema；6 次 `/api/query` 全部返回 200

在 Web 中对问题“What are Maria’s dogs’ names?”（conv-41）发起真实查询：初始 Top-K 只找到 Shadow，Graph 沿 NEXT_STATE 一跳找回记录 Coco 的 `mem-conv41-450`，最终答案为“Shadow and Coco”，并回链 session_31 / D31:4 与 session_30 / D30:1，展示了从检索、补选到答案引用的完整证据链。

4.3. 第一次运行（v1）：多模型对照

v1 在旧版答案提示词（最大输出 1024 token）下，用 DeepSeek Flash 与 GLM-4-Flash 对 340 道测试题各跑五组方法；GLM-4.7 因接口限流未形成有效成绩。

模型	方法	F1	EM	Memory 召回	平均候选	平均输入 Token	拒答	错误
DeepSeek Flash	Full Text	0.5043	0.2324	—	—	19,983.2	61	0
	Embedding K5	0.4069	0.2029	59.20%	5.00	639.0	113	0
	Hybrid K5	0.3976	0.2029	59.92%	5.00	669.1	109	0
	K5 + 补检	0.4075	0.2088	64.97%	6.81	828.8	101	0
	Hybrid K10	0.4243	0.2059	67.45%	9.99	1,082.4	98	0
GLM-4-Flash	Full Text	0.0405	0.0235	—	—	20,010.5	0	300
	Embedding K5	0.3659	0.1882	59.20%	5.00	678.4	81	33
	Hybrid K5	0.3663	0.1912	59.92%	5.00	706.3	103	19
	K5 + 补检	0.3647	0.1941	64.97%	6.81	868.2	89	30
	Hybrid K10	0.3491	0.1765	67.45%	9.99	1,127.4	73	48

v1 暴露了三个问题，直接决定了 v2 的改进方向：（1）补检召回提升，但答案收益不显著：两个模型的补检召回都从 59.92% 升到 64.97%，但 DeepSeek 的 F1 只提高 0.99 个百分点（26 升 15 降，95% 置信区间 [−0.69, +2.69] 跨 0），GLM 反而下降 0.15 个百分点；（2）旧提示词下答案冗长、粒度不对，多给的证据有时反而把模型带偏；（3）GLM-4-Flash 接口不稳定：Full Text 组 300/340 次协议错误，其余各组 19 – 48 次错误计 0 分，K10 甚至低于 K5，难以作为语义能力的证据。因此 v2 改进答案约束、引入 Graph 自适应补选，并统一使用稳定的 DeepSeek Flash 重测全部方法。

4.4. 最终运行（v2）：测试集主要结果

方法	F1	EM	Memory 召回	平均候选	平均输入 Token	拒答
Full Text	0.5692	0.3500	—	—	20,139.2	43
Embedding K5	0.4677	0.2794	59.20%	5.00	795.0	98
Hybrid K5	0.4611	0.2735	59.92%	5.00	825.1	98
K5 + 补检	0.4823	0.3000	64.97%	6.81	984.8	86
Hybrid K10	0.5014	0.2941	67.45%	9.99	1,238.4	79
Forced Graph	0.4887	0.2971	66.55%	9.75	1,263.2	83
Adaptive Graph	0.4837	0.2912	63.89%	6.36	948.8	83

v2 共 2,720 行作答记录均无接口错误（K5 + 补检有 3 题按补检规则跳过生成、计 0 分）。Graph 两组与其基线 Fixed K5 在同一批请求中运行，Fixed K5 的 F1 为 0.4573，与独立运行的 Hybrid K5（0.4611）相差 0.004，可视为运行间波动的量级。Full Text 精度最高，但每题输入约 2 万 token，是 Adaptive 的约 21 倍；在检索类方法中，K10 的 F1 最高，补选方法位于 K5 与 K10 之间，且输入 Token 明显少于 K10。

4.5. 配对比较与开发集确认

比较（后者 − 前者）	测试 ΔF1	升 / 降	测试 95% CI	开发 ΔF1	升 / 降	开发 95% CI
Hybrid K5 → K5 + 补检	+0.0212	21 / 10	[+0.0034, +0.0400]	—	—	—
Fixed K5 → Forced Graph	+0.0314	26 / 10	[+0.0126, +0.0515]	+0.0186	55 / 32	[+0.0071, +0.0304]
Fixed K5 → Adaptive Graph	+0.0264	14 / 0	[+0.0122, +0.0432]	+0.0150	19 / 0	[+0.0079, +0.0232]
Forced → Adaptive Graph	−0.0050	10 / 12	[−0.0171, +0.0066]	−0.0036	32 / 36	[−0.0124, +0.0050]
Hybrid K10 → Adaptive Graph	−0.0177	20 / 26	[−0.0396, +0.0043]	—	—	—
Hybrid K5 → Hybrid K10	+0.0403	37 / 15	[+0.0182, +0.0629]	—	—	—
Hybrid K10 → Full Text	+0.0678	113 / 68	[+0.0191, +0.1171]	—	—	—

三点结论：（1）补选在同一版本内稳定有效：补检与两种 Graph 相对 K5 的提升置信区间均不含 0，开发集 650 题上同样成立。（2）Adaptive 与 Forced 精度相当，但更稳、更省：两者差值在两个数据集上都跨 0；Adaptive 在两个集合上都没有下降题，而 Forced 在开发集上有 32 题下降，Adaptive 的输入 Token 少约 25%。（3）补选没有超过直接取 K10：K10 比 Adaptive 高 1.8 个百分点（置信区间刚好跨 0），但多用约 30% 的输入；补选的价值是以更少上下文拿回 K5 → K10 一半以上的收益，而不是抬高精度上限。

4.6. 提示词版本效应（v1 → v2）

方法（DeepSeek，同题配对）	v1 F1	v2 F1	ΔF1	升 / 降	95% CI
Full Text	0.5043	0.5692	+0.0649	117 / 79	[+0.0292, +0.1022]
Embedding K5	0.4069	0.4677	+0.0608	74 / 37	[+0.0323, +0.0901]
Hybrid K5	0.3976	0.4611	+0.0635	77 / 27	[+0.0371, +0.0906]
K5 + 补检	0.4075	0.4823	+0.0748	84 / 30	[+0.0473, +0.1045]
Hybrid K10	0.4243	0.5014	+0.0771	93 / 37	[+0.0485, +0.1072]

新版提示词使所有方法的 F1 提升 6 – 8 个百分点，幅度大于任何补选方法本身的贡献，因此不能把 v1 到 v2 的总提升归功于补选。补检的变化最能说明这一点：两次运行的补检代码与结果完全相同（225 题补入候选，其中 36 题证据召回提高），但在这 36 题上，v1 有 11 题变好、3 题变差，v2 有 10 题变好、0 题变差；补检相对 K5 的提升也从 v1 不显著的 +0.99 个百分点变为 v2 显著的 +2.12 个百分点（95% 置信区间 [+0.34, +4.00]）。答案约束到位后，补回的证据才稳定转化为答案质量。

4.7. 补选召回消融

不调用模型的确定性消融（Hybrid K=5，每题最多新增 5 条）显示：Memory 召回从不补选的 59.92% 依次升到 1 – 4 跳的 61.89%、63.66%、65.30%、65.45%，3 跳后基本饱和，而平均候选从 5.00 增至 9.46；3 跳时 32 题召回提高、没有下降题。图遍历只在本地进行，只有被选中的新增 Memory 才进入答案模型的上下文。

4.8. 案例分析

正例　“What are Maria’s dogs’ names?” 的初始 Top-K 只含记录 Shadow 的 Memory；Graph 沿同槽位的 NEXT_STATE 边找回记录 Coco 的 Memory，答案由“Shadow”补全为“Shadow and Coco”，F1 提升到 1.0，两条引用都能回链原文。测试集中“What are the names of James’s dogs?”同理，从拒答变为完整答出三只狗的名字。

边界　召回更多证据不保证答案更好：强制扩图的下降题中，有的把明确日期答成“Yesterday”，有的把完整原因压缩成片段，有的被无关证据带偏时间锚点。这正是自适应策略只在拒答或复数姓名题时才扩图的原因，也是报告始终把证据召回与答案 F1 分开呈现的原因。

5. 局限与结论

5.1. 局限

- 数据为自定的 4 开发 / 2 测试划分，单轮问答，不能外推到开放域或生产对话；测试题在开发过程中已暴露，开发集确认部分缓解了这一问题。
- 最终运行只使用 DeepSeek Flash；GLM 只有旧版提示词下的数据。
- 课程未提供端到端的关系 / Effective Memory 金标；小型关系回归不等同于真实问答中的状态准确率。
- 英文 token F1 / EM 会惩罚近义表达、额外解释与日期写法，不等于人工判断的语义正确率；温度为 0 时同一配置两次请求仍有约 0.004 的波动。
- Graph 是有界结构化遍历，不是开放式因果推理 Agent，也未实现 Memory 压缩。

5.2. 后续工作

- 在更多模型上复核补选效果，并尝试“按需补选 + 更大 K”的组合，检验能否在 K10 的成本以内超过 K10。
- 引入人工语义评分或 LLM Judge，并为关系判断构建小规模端到端金标，补充 token F1 / EM 的不足。

5.3. 结论

项目完成了 SRS 中的结构化 Memory、动态关系分析、有效记忆选择、有据作答、Web 可视化与可复现评测，并把中期规划的“初始检索后补选”落地为查询驱动补检与 Graph Memory 自适应补选。在完全相同的数据与 Memory 上，两次运行、七种方法、测试集与开发集的逐题配对实验表明：补选在同一版本内稳定提升 F1（测试集 +2.1 至 +3.1 个百分点），自适应补选在两个数据集上都没有让任何题变差，并比强制扩图节省约 25% 的上下文；但它没有超过直接取 K10，Full Text 仍是精度上限。更大的提升来自答案约束的改进（+6 至 +8 个百分点）。动态记忆系统的价值因此不在于“始终取回更多”，而在于以可追溯、可控的成本在证据不足时有目的地补全局部记忆，并在精度与上下文成本之间给出可选择的折中。

参考　Maharana et al. Evaluating Very Long-Term Conversational Memory of LLM Agents. ACL 2024（LoCoMo）；DimMem: Dimensional Structuring for Efficient Long-Term Agent Memory；MRAgent: Memory is Reconstructed, Not Retrieved。

代码与结果　仓库 `dynamic-memory-system`：合并结果见 `docs/results/results-v1-v2.md`，原始汇总见 `evaluation/results/`；复现脚本为 `scripts/rerun_final.sh` 与 `scripts/consolidate_results.py`。